SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

1.0 Introduction:

SentimentFlow is a cutting-edge platform designed to automate and optimize customer service workflows using advanced AI and Large Language Models (LLMs). It bridges the gap between raw user feedback and actionable business intelligence by transforming unstructured conversational data into a structured, prioritized queue of tasks.

1.1 Purpose:

This system analysis document highlights the technical scope and design-related decisions behind the SentimentFlow development. The key elements of the system that are covered are:

1. **Architecture:** A centralized FastAPI backend and Streamlit frontend interacting with a local SQLite database.
2. **Data Flows:** How user messages flow from the Customer Portal, through the AI reasoning "Brain", into the database, and finally to the Merchant Dashboard.
3. **Model Process:** The integration of the Google Gemini AI to analyze sentiment, urgency, and generate case summaries.
4. **Role of Reference:** This document acts as a reference for developers to understand the API endpoints, AI schema enforcement, and database architecture.

1.2 Background:

Currently, businesses face a chaotic influx of unstructured customer data, such as reviews, messages, and support tickets. Evaluating this sentiment and urgency relies heavily on tedious manual labor, making it difficult to maintain a scalable support infrastructure.

Previous Version / Status Quo:

Before SentimentFlow, resolving customer complaints required authorized personnel to review every single interaction manually from scratch, lacking an intelligent mechanism to categorize and prioritize critical issues instantly.

Changes in Major Architectural Components:Introduction of an AI "Brain" module capable of dissecting content to understand not just sentiment, but intent, urgency, and key topics, automatically generating suggested resolutions.

1.3 Target Stakeholder

Stakeholders	Roles	Expectations
Customer	Submits messages and feedback via the Streamlit Customer Portal.	Rapid routing of issues, clear FAQ responses, and seamless submission of feedback
Merchants / Admins	Manages workflows and reviews AI decisions via the Merchant Dashboard.	A live dashboard visualizing sentiment trends, active complaints, and AI-suggested resolutions to reduce manual workload.
Development Team	Builds the platform and maintains the integrations.	Clear API contracts via FastAPI, well-documented schemas, and straightforward database management.
QA Team	Validates the system behavior across user roles.	Defined fallback behaviors for ambiguous AI responses and thread-safe database interactions.

2. System Architecture & Design

2.1 High Level Architecture

2.1.1 Overview:

Type	Details
System	Web Application (Streamlit Frontend + FastAPI Backend)
Architecture	Client-Server Architecture with Local Database Persistence

SentimentFlow is structured as a client-server-based system. The Streamlit frontend handles session routing between the "Customer Portal" and "Merchant Dashboard". It communicates via HTTP requests to a centralized FastAPI backend (http://localhost:8000). The backend houses distinct endpoints for processing messages, updating tickets, and submitting feedback, which are all persisted in a local sentimentflow.db SQLite database.

2.1.2 LLM as Service Layer

The SentimentFlowEngine acts as the core reasoning layer, integrating the gemini-3-flash-preview model via the Google genai SDK.

2.1.3 Dependency Diagram (Description of Flow):

- **Context Window:** The system injects unstructured user messages, optionally alongside customer_id and order_id, into the LLM context.

- **Parsing and Constraints:** The AI's response is strictly enforced using Pydantic schemas (e.g., UnifiedResponseSchema, DataSchema, VerificationSchema) to ensure the backend receives cleanly formatted JSON containing sentiment scores, urgency levels, and suggested actions.
- **Fallback:** If the AI is unsure of the user's intent, the schema handles failures gracefully by returning an is_ambiguous boolean flag.

2.1.4 Sequence Diagram (Customer Complaint Flow):

1. The user submits a message via the Streamlit Customer Portal.
2. Streamlit sends a POST request to the backend /process endpoint.
3. FastAPI routes the payload to the SentimentFlowEngine (Gemini API).
4. The Brain parses the intent and extracts structured sentiment/urgency data.
5. FastAPI calls database.py to either query FAQs or log a new Ticket in SQLite.
6. A success response with the structured data is sent back to the user interface.

2.2 Technological Stack

1. **Frontend:** Streamlit (Python)
2. **Backend:** FastAPI & Uvicorn (Python)
3. **Database:** SQLite (Local Persistence)
4. **AI Engine:** Google Gemini (gemini-3-flash-preview), google-genai SDK, Pydantic (Schema Validation)

2.3 Key Data Flows

2.3.1 Data Flow Diagram (DFD) Structure:

Data originates from the user on the Streamlit interface and is sent via the requests library to the FastAPI backend. The FastAPI main.py routes the payload to brain.py (external LLM service) for processing, receives the structured output, and forwards the relevant fields (sentiment, message, action) to database.py for storage in the local SQLite database. Finally, the backend serves the data to the Merchant Dashboard for visualization.

2.3.2 Normalized Database Schema (3NF):

The database uses a relational structure managed locally in sentimentflow.db.

- **Tickets Table:** Stores ticket_id, customer_id, order_id, message, sentiment, urgency_level, suggested_action, status, and created_at.
- **Feedback Table:** Stores user feedback, tracking customer_id, message, sentiment_score, and suggested_action.
- **Additional Tables:** Orders for transaction lookups, Todos for task management, and

knowledge_base for FAQs.

3. Functional Requirements & Scope

3.1 Minimum Viable Product (MVP):

#	Features	Description
1	**Message Processing**	The /process endpoint utilizes Gemini AI to classify messages dynamically, either providing an FAQ answer or creating a support ticket.
2	**Dashboard Resolution**	Merchants can review active tickets via /tickets and update their status (e.g., Approve & Resolve) using a PATCH request.
3	**Sentiment Analysis**	Users submit thoughts via /feedback, which the AI scores (e.g., positive, negative, anger, joy). Results are visualized on a live dashboard.

3.2 Non-Functional Requirements (NFRs)

Quality	Requirements	Implementation
Reliability	The AI must return predictable data structures to prevent backend crashes.	Strict enforcement using Pydantic schemas (UnifiedResponseSchema) to format model output.
Cost & Latency	Analysis of tickets must be rapid and keep inference costs manageable.	Selection of the lightweight gemini-3-flash-preview model specifically for high-speed reasoning.
Thread Safety	The local database must handle concurrent requests from multiple users.	SQLite is instantiated with check_same_thread=False to allow FastAPI asynchronous write operations.
Security	Admin functions should be shielded from regular users.	A basic session state routing toggle (is_authenticated) shields the Merchant Dashboard.

3.3 Out of Scope / Future Enhancements

- **Multimodal Vision Integration:** Allowing customers to upload photos of broken items for Gemini to visually analyze and verify damage.
- **Deployment of external/cloud database implementations:** (currently relies entirely on a local SQLite file).
- **Complex multi-factor authentication systems** for the admin portal.

4. Monitor, Evaluation, Assumptions & Dependencies

4.1 Technical Evaluation:

4.1.1 A/B Prompt Testing:
When updating the system prompt for Gemini (e.g., to improve sentiment accuracy), the new prompt is tested against a shadow queue of past tickets to compare the JSON outputs against the Golden Version before going live.

4.1.2 Emergency Rollback (ER) & Golden Release:
If Gemini API changes its schema output unexpectedly, causing the JSON validator to fail >5% of the time, the system will trigger an Emergency Rollback, bypassing the AI entirely and routing all raw text directly to the Admin Dashboard (Legacy Mode) until fixed.

4.1.3 Priority Matrix:

1. **P0 (Critical):** Gemini API returns 401 (Auth Error) or Streamlit UI crashes. Action: Immediate developer intervention.
2. **P1 (High):** SQLite Database locks due to concurrency, preventing order verification. Action: Implement retry logic and alert backend lead.
3. **P2 (Medium):** Misclassification of complaints as FAQs. Action: Log errors and refine the classification prompt during the next sprint.

4.2 Monitoring:

1. Python logging library will capture API latency times, token usage per request, and the pass/fail rate of the JSON schema validation.

4.3 Assumptions:

1. The system assumes the FastAPI backend is running concurrently with the Streamlit frontend locally at http://localhost:8000.
2. The environment assumes the presence of a valid .env file containing a GEMINI_API_KEY for the AI processor to function.

4.4 External Dependencies:

Tools	Purpose	Risks
Google Gemini API (google-genai)	Core LLM reasoning engine for extracting intent, scoring sentiment, and returning Pydantic structured output.	High risk of failure if API key is missing or rate limits are hit. Handled via VerificationSchema and ambiguity fallback flags.
FastAPI & Uvicorn	Provides the RESTful API endpoints and backend server capabilities.	Medium risk; requires virtual environment management.

5. Project Management & Team Contributions

5.1 Project Timeline (10 Days to Hackathon):

- **Days 1-2:** Schema Design, PRD/SAD Documentation, GitHub Repo Setup.
- **Days 3-5:** Backend Development (SQLite DB setup, Gemini API Python integration, Prompt Engineering).
- **Days 6-8:** Frontend Development (Streamlit UI for Customer and Admin).
- **Days 9-10:** QA Testing, Edge Case Mitigation, Pitch Deck Creation, and Video Recording.

5.2 Team Members Role:

- **Jian Jia & Xiao Yan:** Product Leads (Streamlit UI Development, QA Testing, Pitch Formatting).
- **Tim:** AI Logic Architect (Gemini API Integration, Prompt Engineering, JSON Output Formatting).
- **Andy:** Data Engineer (SQLite Architecture, Backend Query Functions, Security/.env Management).